

Week 1 — Foundations: Cleaning & Exploring the Applicant Dataset

Detailed Task Guide for Interns (Beginner-Friendly)

Objective: Practice data cleaning, wrangling, and basic exploration using Pandas — the single most important skill in data science, since real-world data is almost never clean.

Why this task? Every data science project — no matter how advanced — starts with messy, imperfect data. Missing values, typos, duplicate entries, and inconsistent formatting are normal, not exceptions. Before you can analyze anything or build a model, you must learn to clean and trust your data. This week, you'll simulate exactly what a real NextGenLearners data analyst would face if asked to make sense of raw applicant records.

Step 1: Set Up Your Environment

Recommended tool: Google Colab (free, works in your browser, no installation needed) — or Jupyter Notebook if you already have Python installed locally.

Why a notebook instead of a plain script? A notebook lets you run code in small chunks (cells) and mix code with written explanations (markdown). This mirrors exactly how real data scientists document their work — every important decision should be explained in words, not just buried in code. This task specifically requires that documentation.

Setup steps:

1. Go to colab.research.google.com and create a new notebook (or open Jupyter Notebook locally).
2. Rename it something clear, like `Week1_Data_Cleaning_YourName.ipynb`.
3. In the first cell, import the library you'll need:
4. `import pandas as pd`
5. Create a markdown cell above it (in Colab/Jupyter, you can change a cell type to "Markdown") with a short title and your name — this is the start of your documented report.

Step 2: Get (or Create) `applicants.csv`

If your mentor provides the file: Upload it into Colab using the file upload button on the left sidebar, or use:

```
from google.colab import files
uploaded = files.upload()
```

If you need to create your own mock dataset: Build a CSV with these columns and deliberately include messy data so you have real problems to practice fixing:

Column	What it should contain	Example of "messy" version to include
Applicant Name	Full name	Some blank, some extra whitespace (`" Ali Khan "`)
Email	Email address	Some missing, some with inconsistent casing
Phone	Phone number	Stored as text with dashes/spaces (`"0301-1234567"`)
Domain Applied	One of the 5 tracks	Inconsistent casing/spelling (`"web dev"`, `"Web Development"`, `"WEB DEV"`)
University	University name	Some blank, some inconsistent abbreviations
Application Date	Date applied	Stored as plain text, inconsistent formats (`"12/05/2026"` vs `"2026-05-12"`)
Status	Under Review / Selected / Rejected	Some blank, some lowercase/uppercase inconsistently

Tip: Aim for at least 60–100 rows, with roughly 10–15 rows containing at least one issue each (missing value, duplicate, bad formatting), so your cleaning process has real work to do. You can generate fake-but-realistic names/emails using Python's `Faker` library if you want to avoid manually typing everything:

```
from faker import Faker
fake = Faker()
fake.name(), fake.email()
```

Load your CSV into a DataFrame:

```
df = pd.read_csv('applicants.csv')
```

Step 3: First Look — Understand Before You Clean

Never start cleaning blindly. First, explore what you're working with:

```
df.shape           # how many rows and columns?
df.head()          # preview the first 5 rows
df.info()          # data types + non-null counts per column
df.isnull().sum() # exact count of missing values per column
df.duplicated().sum()  # how many exact duplicate rows exist
```

Write a markdown cell summarizing what you observe, for example: "The dataset has 85 rows and 7 columns. `Email` has 6 missing values, `University` has 4 missing values, there are 3

exact duplicate rows, and `Application Date` is currently stored as text (object type), not as an actual date."

Why this step matters: This is your "before" snapshot. You'll compare it to your "after" state at the end for your data quality summary — and it forces you to actually look at the data instead of applying cleaning steps blindly.

Step 4: Handle Missing Values (Column by Column, With Justification)

The key lesson this week: There is no single "correct" way to handle missing data — the right choice depends on the column and what the data is used for. You must decide and explain your reasoning for each column, not apply one blanket rule to everything.

Go column by column and decide:

- **Applicant Name missing?** This is critical identifying information — a row without a name is not very useful. Consider **dropping** these rows:
- `df = df.dropna(subset=['Applicant Name'])`
- **Email missing?** Also fairly critical (used for follow-up). You could drop these rows too, OR fill with a placeholder like `"not_provided@example.com"` if you want to keep the row for other analysis purposes. Justify whichever you choose.
- **University missing?** Less critical than name/email. Instead of dropping the whole row, **fill** the gap with a placeholder:
- `df['University'] = df['University'].fillna('Not Specified')`
- **Status missing?** If status wasn't recorded, it likely means the application hasn't been processed yet. Fill with a sensible default:
- `df['Status'] = df['Status'].fillna('Under Review')`

For every decision, write a short markdown note explaining WHY, e.g.: *"I dropped rows with missing Applicant Name because a nameless application isn't usable for outreach. I filled missing University with 'Not Specified' instead of dropping, since it's not essential to determining application status."*

Why this matters for evaluation: Mentors are specifically checking that you made *justified* decisions, not that you memorized one universal "always drop" or "always fill" rule. Real data cleaning requires judgment.

Step 5: Remove Duplicate Applicants

The problem: Sometimes the same person accidentally applies twice (form resubmission, double-click, etc.), creating duplicate rows.

```
# Check exact duplicates first
df.duplicated().sum()

# Remove exact duplicate rows
df = df.drop_duplicates()
```

Go one level deeper — check for near-duplicates too. Someone might submit the same application twice with slightly different formatting (e.g., extra whitespace in their name), which `drop_duplicates()` alone won't catch. A smarter approach: check for duplicates based on a cleaned version of Email (a more reliable unique identifier than Name):

```
# Check for duplicate applications by the same email, keeping only the first
entry
df = df.drop_duplicates(subset=['Email'], keep='first')
```

Explain in markdown: how many duplicate rows you found and removed, and why you chose to check by email rather than by full row or by name

Step 6: Standardize Text Fields

The problem: The same real-world value can appear in many different text forms "`web dev`", "`Web Development`", and "`WEB DEV`" are all the same domain, but a computer sees them as three different strings unless you standardize them.

Step-by-step approach:

1. **Strip extra whitespace** (invisible characters that mess up matching):
2. **Standardize casing** — decide on one consistent format (e.g., Title Case for names, lowercase for matching/grouping purposes):
3. **Map inconsistent domain spellings to one standard value.** This is the most important standardization step. Build a mapping dictionary of all the messy variations you find to their correct final label:

```
domain_mapping = {
    'web dev': 'Web Development',
    'webdev': 'Web Development',
    'web development': 'Web Development',
}
df['Domain Applied'] = df['Domain Applied'].map(domain_mapping)
```

   Important: After mapping, run `df['Domain Applied'].unique()` again to confirm only the 5 correct domain names remain — if you see `NaN` values appear, it means some messy variation wasn't caught in your dictionary, and you need to go back and add it.

10. **Standardize the Status column** the same way (e.g., make sure "`selected`", "`Selected`", "`SELECTED`" all become one consistent value):

```
df['Status'] = df['Status'].str.strip().str.title()
```

Why this matters: If you skip this step, later analysis (like "how many people applied to Web Development?") will be wrong — because the computer will count "`Web Dev`" and "`web development`" as two separate categories instead of recognizing them as the same thing.

Step 7: Fix Data Types (Especially Dates and Phone Numbers)

The problem: A date or phone number that's stored as plain text *looks* right to a human but can't be sorted, filtered, or calculated on correctly by code until it's converted to the proper type.

Convert Application Date to a real date type:

```
df['Application Date'] = pd.to_datetime(df['Application Date'],
errors='coerce')
```

Why `errors='coerce'`? If any date is in a broken/unreadable format, this tells Pandas to convert it to `NaT` (Pandas' version of "missing date") instead of crashing your whole notebook. After running this, check `df['Application Date'].isnull().sum()` again — if new missing values appeared, those are the rows where the original date format couldn't be understood, and you should decide how to handle them (drop, or investigate manually).

Clean up Phone numbers by removing dashes/spaces so the format is consistent (even if you keep it as text, since phone numbers with leading zeros shouldn't be converted to actual numbers):

```
df['Phone'] = df['Phone'].str.replace('-', '', regex=False).str.replace(' ',
'', regex=False)
```

Verify your fixes worked:

```
df.info()
```

Confirm `Application Date` now shows as `datetime64` type instead of `object` (text).

Step 8: Write the Data Quality Summary

This is your final deliverable within the notebook — a markdown section (not code) summarizing your entire cleaning process, written clearly enough that someone who never looked at the raw data could understand what you did and why.

Structure it like this:

```
Data Quality Summary

Starting dataset: 85 rows, 7 columns

Issues found:
- 6 missing values in Email, 4 in University, 3 in Status
- 3 exact duplicate rows, plus 2 additional duplicate applications
  identified by matching Email
- Domain Applied had 9 different spelling/casing variations for
  only 5 real domains
- Application Date was stored as text, not as an actual date type
- Phone numbers contained inconsistent dash/space formatting
```

```
Actions taken:
- Dropped 2 rows with missing Applicant Name (unusable without
identification)
- Filled missing University with "Not Specified" (non-critical field)
- Filled missing Status with "Under Review" (default for unprocessed
applications)
- Removed 5 duplicate rows total (3 exact, 2 by matching Email)
- Standardized all Domain Applied values into 5 consistent categories
- Converted Application Date to proper datetime format
- Cleaned Phone number formatting for consistency

Final dataset: 78 rows, 7 columns, no missing values in critical
fields, no duplicates, consistent formatting throughout.
```

Adjust the numbers to match your actual dataset — this is just the format/structure to follow.

Step 9: Export the Cleaned Dataset

```
df.to_csv('applicants_cleaned.csv', index=False)
```

Why `index=False`? Without it, Pandas adds an extra unnamed column with row numbers to your CSV, which isn't needed and looks messy when someone else opens the file.

Download it from Colab (Files sidebar → right-click → Download) or confirm it saved correctly if working locally.

Step 10: Final Checklist Before Submitting

- [ ] Notebook runs top-to-bottom without errors (in Colab/Jupyter: Runtime → Run All, and confirm no red error cells)
- [ ] Every cleaning decision (especially missing value handling) has a markdown explanation next to it — not just code with no explanation
- [ ] `isnull()`, `sum()`, `drop_duplicates()`, `fillna()`, `astype()`/`pd.to_datetime()`, and `str.strip()`/`str.lower()` are all used somewhere in the notebook
- [ ] `Domain Applied` column has exactly 5 clean, consistent values (verified with `.unique()`)
- [ ] `Application Date` is confirmed as a real datetime type (checked with `df.info()`)
- [ ] No exact or near-duplicate applicants remain in the final dataset
- [ ] A clear "Data Quality Summary" markdown section exists near the end of the notebook
- [ ] `applicants_cleaned.csv` is exported and included in the GitHub repo alongside the notebook
- [ ] No manual editing was done in Excel — all fixes exist as code in the notebook (so someone else could re-run it from scratch)

Suggested Daily Breakdown

Day	Focus

Day 1	Set up Colab/Jupyter, get or generate `applicants.csv`, load and do first exploration (Step 3)
Day 2	Handle missing values column by column, with written justification (Step 4)
Day 3	Remove exact and near-duplicate rows (Step 5)
Day 4	Standardize text fields, especially the Domain Applied mapping (Step 6)
Day 5	Fix data types for dates and phone numbers (Step 7)
Day 6	Write the Data Quality Summary, do a final review pass (Steps 8)
Day 7	Export cleaned CSV, run notebook top-to-bottom to confirm no errors, push to GitHub

Common Mistakes to Avoid

1. **Cleaning without exploring first** — always run `.info()`, `.isnull().sum()`, and `.head()` before touching anything, so you know what you're actually fixing.
2. **Applying one rule to every column** (e.g., dropping all rows with any missing value) — this often deletes far more data than necessary. Handle each column thoughtfully.
3. **Forgetting to re-check after standardizing** — after mapping domain names, always run `.unique()` again to confirm nothing slipped through as `NaN` or an unmapped value.
4. **Editing the CSV manually in Excel** — this defeats the purpose of the task, isn't reproducible, and mentors will ask you to demonstrate the notebook runs cleanly from the original raw file.
5. **Skipping markdown explanations** — code alone doesn't show your reasoning. A big part of this task's grade is *why* you made each decision, not just *what* you did.
6. **Converting dates without `errors='coerce'`** — a single unreadable date format can crash `pd.to_datetime()` entirely if you don't handle errors gracefully.
7. **Confusing exact duplicates with near-duplicates** — `drop_duplicates()` alone only catches rows that are 100% identical across all columns; always also check for duplicates by a key field like Email.

Helpful Free Resources

- Pandas official 10-minute intro: pandas.pydata.org/docs/user_guide/10min.html
- Handling missing data: Pandas – Working with missing data
- `pd.to_datetime()` docs: pandas.pydata.org – to_datetime
- Faker library (for generating mock data): faker.readthedocs.io
- Google Colab quickstart: colab.research.google.com

Deliverables (Reminder)

1. `Week1_Data_Cleaning_YourName.ipynb` — fully documented notebook with markdown explanations throughout
2. `applicants_cleaned.csv` — the final cleaned dataset
3. Both files pushed to a public GitHub repo

Evaluation Criteria

Criteria	What Mentors Are Checking

Correct identification and handling of data issues	Were missing values, duplicates, inconsistent text, and wrong data types all correctly found and fixed?
Clarity of markdown explanations	Does every major decision have a clear, sensible written justification?
Reproducibility	Does the notebook run top-to-bottom with no errors, using only code (no manual CSV edits)?
Clean final dataset	Is `applicants_cleaned.csv` genuinely clean — consistent formatting, no duplicates, correct types?